SCSE2233 SOFTWARE DESIGN & ARCHITECTURE

MOCK FINAL EXAMINATION — SEMESTER 2, 2025/2026

ANSWER SCHEME, GRADING RUBRIC AND EXAMINER NOTES

Self-mark only AFTER attempting the paper under exam conditions (3 hours, closed book, diagrams by hand). Marks shown in [brackets] inside each model answer.

SECTION A — MODEL ANSWERS

QUESTION 1 (16 marks) — Software Design Management, Leadership & Ethics

1(a) WBS + Linear Responsibility Chart (6 marks)

Model WBS (list format; graphical tree equally acceptable). The WBS must contain TASKS ONLY (scope accuracy), cover the whole design project (completeness), and break down to effort-estimable work packages (level of detail).

```
1      CampusRide Design Project
1.1    Architectural Design
1.1.1  Evaluate Alternative Architectures
1.1.1.1 Prioritize Design Objectives (availability, performance, security)
1.1.1.2 Formal Review of Evaluations
1.1.2  Select Architectural Design
1.1.2.1 Make Initial Selection
1.1.2.2 Formal Review to Select Final Design
1.2    Detailed Design
1.2.1  Design Live-Tracking & ETA Prediction Components
1.2.2  Design Route-Planning & Seat-Availability Components
1.2.3  Design Driver Console & Admin Analytics Components
1.2.4  Design Role-Based Security (students / drivers / admin)
1.3    Documentation
1.3.1  Write Architectural Design Document
1.3.2  Formal Review (Architectural Design)
1.3.3  Finalize Architectural Design Document
1.3.4  Write Detailed Design Document
1.3.5  Formal Review (Detailed Design)
1.3.6  Finalize Detailed Design Document
```

Linear Responsibility Chart — rows = WBS tasks, columns = the FIVE staff members, cells = responsibility codes with the legend 1 = Primary responsibility, 2 = Support development role, 3 = Review role. Sample rows:

WBS #	Task	S1	S2	S3	S4	S5
1.1.1	Evaluate Alternative Architectures	1	2		3	
1.1.2	Select Architectural Design	1		2	3	3
1.2.1	Live-Tracking & ETA Components		1	2		3
1.2.4	Role-Based Security			1	2	3
1.3.1	Write Architectural Design Doc.	3			1	2
1.3.4	Write Detailed Design Doc.	3			2	1

Marking: hierarchical decomposition with numbered outline levels [2]; tasks only — no durations, no precedence, no personnel skills inside the WBS (scope accuracy) [1]; full design lifecycle covered — architectural design, detailed design, documentation with formal reviews (completeness) [1]; work packages small enough to estimate effort (level of detail) [1]; LRC distributes ALL tasks over the five members using the 1/2/3 legend, every task has exactly ONE primary [1].

TRAP TO WATCH: *Putting task durations, arrows, or staff skills INSIDE the WBS violates scope accuracy — the Doctor's slides list this as the first evaluation criterion. A WBS that stops at level 2 (no work packages) loses the level-of-detail mark. In the LRC, two '1' codes on one task = no single owner = mark lost.*

1(b) EVM Calculation (6 marks)

% completed = 4/12 tasks; % scheduled = 7/12 months.

Parameter	Calculation	Result
EV	(4/12) × RM 18,000	RM 6,000 [1]
PV	(7/12) × RM 18,000	RM 10,500 [1]
CV	EV − AC = 6,000 − 7,500	−RM 1,500 [0.5]
SV	EV − PV = 6,000 − 10,500	−RM 4,500 [0.5]
CPI	EV / AC = 6,000 / 7,500	0.80 [0.5]
SPI	EV / PV = 6,000 / 10,500	0.5714 [0.5]
ETC	(BAC − EV) / CPI = (18,000 − 6,000) / 0.80	RM 15,000 [0.5]
EAC	ETC + AC = 15,000 + 7,500	RM 22,500 [0.5]

Interpretation [0.5 each, max 1]: CPI = 0.80 → only RM 0.80 of earned value is received for every RM 1 actually spent — the project is OVER BUDGET. SPI = 0.57 → work is progressing at only 57% of the planned rate — the project is BEHIND SCHEDULE. (EAC shows an estimated RM 4,500 overrun on the RM 18,000 budget.) Calculation marks [5] + interpretation [1] = 6.

TRAP TO WATCH: *Most common loss: using AC instead of PV in SV (SV compares against the SCHEDULE baseline, not spending). Second: computing PV from tasks completed instead of months elapsed. Note the lecture's Sara example prints EAC = ETC + (BAC − EV) on the slide ($16,333); the formula table itself says EAC = ETC + AC — in the real exam FOLLOW THE APPENDIX FORMULA, and state your formula before substituting so method marks are safe either way.*

1(c)(i) Ethics — SECE principle (2 marks)

Best answer — PRODUCT principle ("software engineers shall ensure that their products and related modifications meet the highest professional standards possible"), specifically guideline 3.09: provide REALISTIC quantitative estimates of cost, scheduling, personnel and quality [1]. Reporting partially started tasks as complete falsifies the % completion input to EVM, so every reported figure (EV, CPI, SPI) becomes a deliberate misestimate presented to the client [1]. Also creditable: JUDGMENT principle (maintain integrity and independence in professional judgment; no deceptive practices) or PUBLIC principle (act consistently with the public interest), each with a correct explanation.

1(c)(ii) Leadership — one of the FOUR keys (2 marks)

The four keys taught: Communication, Networking, Motivation, Negotiation. Example using COMMUNICATION [1]: apply the five-step oral-message process — develop rapport with the manager, clearly state the objective (the report must reflect the true 50-50/0-100 completion estimates), transmit the actual EVM results, confirm the manager's understanding of the risk of misreporting, and get a commitment to send the corrected status [1]. Example using NEGOTIATION: principled negotiation — separate the people from the problem, focus on interests (client trust, risk mitigation) rather than positions, generate options (recovery plan, re-baselining) before agreement, insist on objective criteria (the EVM data itself).

TRAP TO WATCH: *Name the key EXACTLY as taught (Communication / Networking / Motivation / Negotiation). Generic answers ('be a good leader') earn zero. Also be ready for the sibling question the Doctor has used twice before: apply TWO Big Five personality dimensions (Agreeableness, Surgency, Adjustment, Conscientiousness, Openness to experience) to your project experience.*

QUESTION 2 (12 marks) — Construction Design: Tabular Design

2(a) Quality attributes (2 marks)

Any TWO of the four taught [1 + 1]: **completeness, correctness, testability, maintainability**. (Completeness/correctness = degree to which the construction design meets the allocated requirements; testability/maintainability = effort required to test / maintain the resulting artifact, both measurable via cyclomatic complexity.)

2(b) Table type + policy count (2 marks)

EEDT (Extended Entry Decision Table) [1] — because the conditions are complete questions with MORE THAN TWO possible values (Membership Tier has THREE values), so Boolean condition entries (LEDT) cannot represent them directly. Number of distinct elementary policies = ∏ Vi = V1 × V2 = 3 × 2 = **6 policies [1]**. (For an LEDT the count would be 2^n; that formula does NOT apply here.)

2(c) The EEDT (5 marks)

Get Delivery Fee & Perk	**P1**	**P2**	**P3**	**P4**	**P5**	**P6**
Membership Tier is	BASIC	BASIC	GOLD	GOLD	PLAT	PLAT
Order Value is	< RM50	≥ RM50	< RM50	≥ RM50	< RM50	≥ RM50
Delivery Fee	RM 5	RM 3	RM 3	RM 1	RM 1	RM 0
Free Perk	NONE	DRINK	DRINK	DESSERT	DESSERT	DESSERT + PRIORITY

Marking: four-section structure — Condition / Condition Entry / Action / Action Entry [1]; both conditions written as complete questions with extended (non-Boolean) entries [1]; all 6 policies present, one column each [1]; delivery-fee row fully correct [1]; perk row fully correct including the double action DESSERT + PRIORITY in P6 [1].

TRAP TO WATCH: *The classic error is drawing this as an LEDT with T/F rows like 'Tier is GOLD? T/F' — that explodes to 2^3 columns with impossible combinations and loses the 'type' marks. Conditions must read as questions ("Membership Tier is"), entries carry the values.*

2(d) Code excerpt implementing the EEDT (3 marks)

```
public class DeliveryPolicy {
  public static void applyPolicy(String tier, double orderValue) {
    double fee;  String perk;
    if (tier.equals("BASIC")) {
      if (orderValue < 50) { fee = 5; perk = "NONE"; }        // P1
      else                 { fee = 3; perk = "DRINK"; }       // P2
    } else if (tier.equals("GOLD")) {
      if (orderValue < 50) { fee = 3; perk = "DRINK"; }       // P3
      else                 { fee = 1; perk = "DESSERT"; }     // P4
    } else { // PLATINUM
      if (orderValue < 50) { fee = 1; perk = "DESSERT"; }     // P5
      else                 { fee = 0; perk = "DESSERT + PRIORITY"; } // P6
    }
    System.out.println("Fee: RM " + fee + ", Perk: " + perk);
  }
}
```

Marking: branching on membership tier [1]; branching on order value inside each tier [1]; all six fee/perk outcomes match the table exactly [1]. Any taught language (Java/C++/C#) accepted.

TRAP TO WATCH: *This is In-Class Exercise #1's skill (write the source code from the given decision table). Cross-check EVERY policy column against your code — one mismatched cell (e.g., P5 giving DRINK) costs the outcome mark.*

QUESTION 3 (12 marks) — Construction Design: PDL, Flow Graph, Cyclomatic Complexity

3(a) PDL (4 marks)

PDL is indented natural language — NO Java/C++ keywords, detailed enough that code can be generated with minimal effort:

```
Set the value of the session status to "failure"
Read the student's matric card
Determine if the card is valid
If the card is not valid
   Display an error message on the kiosk screen
   End the session
If the card is valid
   Retrieve the member record
   Determine the member's outstanding fines
   If the fines exceed RM 10
      Reject the loan session
      Direct the member to the service counter
   Otherwise
      For each book placed on the scanner pad
         Read the book's RFID tag
         Determine if the book is reserved by another member
         If the book is reserved
            Set the book aside and report it on the screen
         If the book is not reserved
            Register the book as a loan
            Set the due date to 14 days from today
      Print a receipt listing every successful loan with its due date
      Set the session status to "success"
Return the value of the session status
```

Marking: card-validation branch with error path [1]; fines check with rejection path [1]; per-book loop containing the reserved/not-reserved branch [1]; receipt printing and success/failure status returned [1].

TRAP TO WATCH: *Writing `if (card == null)` or any programming-language syntax breaks the definition of PDL ('natural language, as opposed to computer languages'). Also do not draw a flowchart here — PDL is textual by definition.*

3(b) Program flow graph (4 marks)

Compound conditions are split into separate predicate nodes. Node list (statement grouping shown for self-checking; any consistent numbering earns full marks):

```
N1  start; declarations (lines 1-5)        N9   rate = 0.50 (default, line 16)
N2  do-body: prompt + read kWh (7-8)       N10  cost = kWh * rate (18)
N3  predicate: kWh < 1       (9)           N11  predicate: kWh > 60 (19)
N4  predicate: kWh > 100     (9)           N12  predicate: type != 3 (19)
N5  read type (10-12)                      N13  cost = cost * 0.90 (20)
N6  switch (type) (13)                     N14  for: i < 2 ? (22)
N7  rate = 0.20 (case 1, line 14)          N15  print receipt; i++ (23)
N8  rate = 0.35 (case 2, line 15)          N16  print total; close; end (25-28)

Edges (22): N1>N2, N2>N3, N3>N2 (T), N3>N4 (F), N4>N2 (T), N4>N5 (F),
N5>N6, N6>N7, N6>N8, N6>N9, N7>N10, N8>N10, N9>N10, N10>N11,
N11>N12 (T), N11>N14 (F), N12>N13 (T), N12>N14 (F), N13>N14,
N14>N15 (T), N15>N14, N14>N16 (F)
```

Marking: do-while loop-back edges from BOTH predicates to the loop body [1]; switch drawn as ONE decision node with THREE outgoing branches converging after the switch [1]; compound

&& drawn as two predicate nodes (or one node explicitly marked as a 2-condition compound) [1]; for-loop with back edge and single exit to the final node [1].

TRAP TO WATCH: *The || in the do-while means BOTH predicate nodes loop back to N2 on TRUE — drawing only one back edge undercounts the edges and your (c) answers will disagree. The for loop needs its back edge N15→N14; forgetting it is the most common single-edge loss.*

3(c) Cyclomatic complexity by TWO approaches (4 marks)

Approach 1 — Mills: C = π + 1 [2]. Conditions: do-while compound (kWh < 1 || kWh > 100) = 2; switch with 3 branches = n − 1 = 2; if compound (kWh > 60 && type != 3) = 2; for loop = 1. π = 2 + 2 + 2 + 1 = 7, so C = 7 + 1 = **8**.

Approach 2 — graph formula: V(G) = e − n + 2p [2]. From the flow graph in (b): e = 22, n = 16, p = 1, so V(G) = 22 − 16 + 2 = **8**. (Or Euler regions: r = 7 enclosed regions + 1 outer = 8.) Both approaches agree: **V(G) = 8** — below McCabe's acceptability threshold of 10.

TRAP TO WATCH: *Three trip-wires the Doctor plants: (1) compound conditions count as TWO units each under Mills — counting the if and the do-while as one unit each gives C = 6, wrong; (2) a 3-branch switch counts n − 1 = 2, not 3; (3) e − n + 2p only matches Mills if the graph SPLITS compound predicates — if you drew single nodes for compound conditions, your e − n + 2 gives 6 and you must say so explicitly or lose the consistency mark.*

SECTION B — MODEL ANSWERS

QUESTION 4 (30 marks) — Creational Design Patterns

4(a) Identify + justify (6 marks)

Project 1 — BUILDER [1]. Justifications [2]: the package is a COMPLEX object constructed through a fixed MULTI-STEP sequence, and the Builder pattern 'separates the construction of a complex object from its representation so that the same construction process can create different representations' (Budget vs Luxury); the client controls the step-by-step creational process (optional steps can be skipped) and the finished product is retrieved only at the end via getProduct() — exactly the Builder's contract, unlike Abstract Factory where product objects are created all at once.

Project 2 — ABSTRACT FACTORY [1]. Justifications [2]: the system must 'create FAMILIES of related or dependent objects without specifying their concrete classes' — each ward line (ICU / General-Ward) is a family of VitalsMonitor + InfusionPump + AlarmPanel; the pattern guarantees consistency within a product family (no ICU monitor with a Ward alarm panel) and lets the client swap ENTIRE families (or add a Pediatric line) without changing client code — additions by extension, obeying OCP.

TRAP TO WATCH: *Two planted decoys: 'package tiers' in Project 1 tempts Abstract Factory — but the scenario stresses STEP-BY-STEP construction of ONE complex object, the Builder discriminator quoted in the slides. 'Different types' in Project 2 tempts Factory Method — but Factory Method creates a SINGLE product type; the family-consistency requirement forces Abstract Factory. Quoting the GoF intent sentence for your chosen pattern is the fastest justification mark.*

4(b) Class diagram — Project 2, Abstract Factory (10 marks)

Draw with UML notation: interfaces in <<interface>> boxes (or italics), dashed realization arrows from concrete classes, dashed <<create>> arrows from factories to products, client associated to the abstract factory only.



Marking: abstract factory interface with the THREE create methods [2]; TWO concrete factories realizing it [2]; THREE abstract product interfaces [2]; SIX concrete products, each realizing its product interface and created by the correct factory [2]; client depends ONLY on the factory + product interfaces, with correct UML arrows (solid-hollow generalization / dashed realization / dashed create dependencies) [2]. If Project 1 chosen instead: Director (TripDirector +construct()), <<interface>> PackageBuilder with buildTransport()/buildAccommodation()/buildActivities()/buildDocuments()/getPackage(), BudgetPackageBuilder and LuxuryPackageBuilder, product TripPackage — same 2+2+2+2+2 split.

TRAP TO WATCH: *The 10-mark diagram bleeds marks through arrows, not boxes: realization must be DASHED with a hollow head; the factory-to-product create arrows are DASHED dependencies; the client must NOT point at concrete classes ('Clients only know about creator and product interfaces!' — the slide annotation). Missing the third product family member (AlarmPanel) silently caps you at 8.*

4(c) Component description (4 marks)

WardEquipmentFactory — the Abstract Factory: declares one creation method per product in the family, so the client can create a whole consistent family through one interface [1]. **ICUEquipmentFactory / GeneralWardEquipmentFactory** — Concrete Factories: each manufactures the complete set of concrete products of ONE family, guaranteeing intra-family consistency [1]. **VitalsMonitor / InfusionPump / AlarmPanel** — Abstract Products: the interfaces the client codes against; 'products need to obey the product interface' [1]. **ICU*/Ward* products + WardStation client** — Concrete Products implement a family's look/behaviour; the Client is configured with a factory (constructor injection) and delegates ALL creation to it, never touching concrete classes [1].

4(d) Java code (10 marks)

```
// Abstract products
interface VitalsMonitor { void display(); }
interface InfusionPump  { void display(); }
interface AlarmPanel    { void display(); }

// Concrete products - ICU family
class ICUVitalsMonitor implements VitalsMonitor {
  public void display() { System.out.println("ICU vitals monitor: 1-sec sampling."); } }
class ICUInfusionPump implements InfusionPump {
  public void display() { System.out.println("ICU infusion pump: high-precision."); } }
class ICUAlarmPanel implements AlarmPanel {
  public void display() { System.out.println("ICU alarm panel: critical-care alarms."); }
}

// Concrete products - General-Ward family
class WardVitalsMonitor implements VitalsMonitor {
  public void display() { System.out.println("Ward vitals monitor: 60-sec sampling."); }
}
class WardInfusionPump implements InfusionPump {
  public void display() { System.out.println("Ward infusion pump: standard."); } }
class WardAlarmPanel implements AlarmPanel {
  public void display() { System.out.println("Ward alarm panel: standard alerts."); } }

// Abstract factory
interface WardEquipmentFactory {
  VitalsMonitor createVitalsMonitor();
  InfusionPump  createInfusionPump();
  AlarmPanel    createAlarmPanel();
}

// Concrete factories (one per family)
class ICUEquipmentFactory implements WardEquipmentFactory {
```

```
  public VitalsMonitor createVitalsMonitor() { return new ICUVitalsMonitor(); }
  public InfusionPump  createInfusionPump()  { return new ICUInfusionPump(); }
  public AlarmPanel    createAlarmPanel()    { return new ICUAlarmPanel(); }
}
class GeneralWardEquipmentFactory implements WardEquipmentFactory {
  public VitalsMonitor createVitalsMonitor() { return new WardVitalsMonitor(); }
  public InfusionPump  createInfusionPump()  { return new WardInfusionPump(); }
  public AlarmPanel    createAlarmPanel()    { return new WardAlarmPanel(); }
}

// Client - delegates ALL creation to its factory
class WardStation {
  private VitalsMonitor monitor;  private InfusionPump pump;  private AlarmPanel
panel;
  public WardStation(WardEquipmentFactory factory) {
    monitor = factory.createVitalsMonitor();
    pump    = factory.createInfusionPump();
    panel   = factory.createAlarmPanel();
  }
  public void equipWard() { monitor.display(); pump.display(); panel.display(); }
}

public class Main {
  public static void main(String[] args) {
    WardStation icuWard  = new WardStation(new ICUEquipmentFactory());
    icuWard.equipWard();
    WardStation genWard  = new WardStation(new GeneralWardEquipmentFactory());
    genWard.equipWard();   // whole family swapped - client code unchanged
  }
}
```

Marking: three product interfaces [2]; six concrete products implementing them [2]; factory interface with three create methods [2]; two concrete factories returning their own family only [2]; client with constructor injection + main demonstrating a family swap without client change [2]. Code must compile as written.

TRAP TO WATCH: *The lecturer's decks write pattern code in C# (new deck) and C++/Java (old decks) — Java is safest in the exam, but keep the METHOD NAMES from your diagram identical in the code; diagram-vs-code name drift is an easy 2-mark deduction under 'code must reflect the diagram'.*

QUESTION 5 (30 marks) — Structural & Behavioral Design Patterns

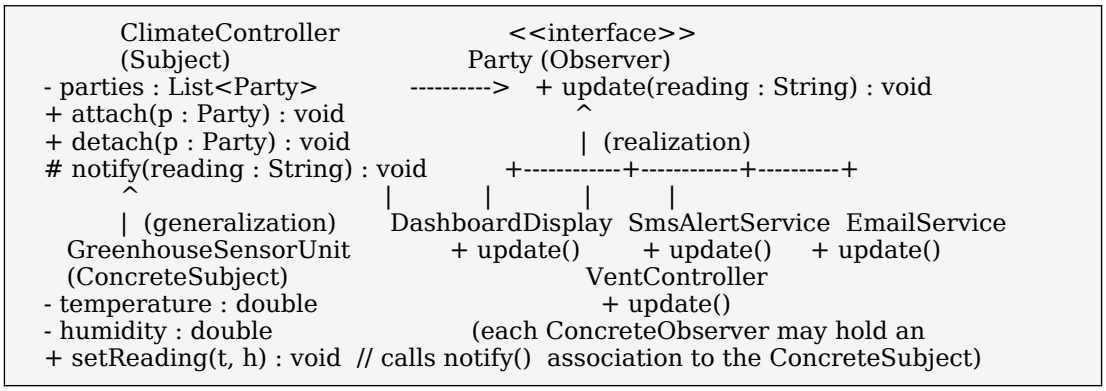
5(a) Identify + category + justify (6 marks)

Project 1 — OBSERVER, a BEHAVIORAL pattern [1]. Justifications [2]: it 'defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically' — one climate controller, many notified parties; observers can be attached/detached dynamically at run time and NEW observer types (mobile push) are added without modifying the subject, since the subject knows only the abstract update interface.

Project 2 — ADAPTER, a STRUCTURAL pattern [1]. Justifications [2]: it 'converts the interface of a class into another interface clients expect', letting 'classes work together that couldn't otherwise because of incompatible interfaces' — trackParcel()/registerParcel()/scanParcel() vs trackCustomsQR()/registerCustomsQR()/scanCustomsQR(); the third-party module is a compiled binary that cannot be modified, so an adapter enables REUSE of existing code without modification on either side.

TRAP TO WATCH: *The category statement is a deliberate trap: a real past paper labelled an Adapter scenario 'behavioral'. Adapter/Composite/Facade = STRUCTURAL (composition of larger structures); Observer/Iterator = BEHAVIORAL (object communication & responsibility). Stating the wrong category forfeits the identification mark even with the right pattern name.*

5(b) Class diagram — Project 1, Observer (10 marks)



Marking: Subject class holding a COLLECTION of observers with attach(), detach(), and notify() [3]; <<interface>> Party/Observer with the abstract update() method [2]; at least THREE concrete observers realizing the interface (dashboard, SMS, email, vent controller) [3]; ConcreteSubject inheriting the Subject and triggering notify() on state change, with correct UML arrows [2]. If Project 2 chosen: Client (DispatchSystem) → <<interface>> ParcelHandler (+trackParcel() +registerParcel() +scanParcel()); CustomsAdapter realizes ParcelHandler AND holds an association to the Adaptee CustomsClearanceLib (+trackCustomsQR() +registerCustomsQR() +scanCustomsQR()) — realization to Target [3], association to Adaptee [3], delegation shown [2], client uses Target only [2].

TRAP TO WATCH: *Use the slides' method names: attach / detach / notify / update. The subject-to-observer link goes from Subject to the Observer INTERFACE (one-to-many '*'), never to concrete observers. In the Adapter variant, the adapter RELATES to the adaptee by association (object adapter, the taught variant) — drawing double inheritance (class adapter) contradicts the taught step-by-step and risks the relationship marks.*

5(c) Component description (4 marks)

ClimateController (Subject) — keeps the registry of observers (a linked list per the slides) and provides the attaching, detaching and notifying mechanism; notify() iterates the registered list and calls each observer's update() [1]. **Party (<<interface>> Observer)** — the abstract update interface every notified party must implement; the subject depends only on this abstraction [1]. **DashboardDisplay / SmsAlertService / EmailService / VentController (ConcreteObservers)** — implement update() with channel-specific reactions (render, send SMS, send email, open vents) [1]. **GreenhouseSensorUnit (ConcreteSubject)** — holds the state of interest (temperature, humidity); when a threshold is crossed in setReading(), it triggers notify(), so all registered parties are updated automatically [1].

5(d) Java code (10 marks)

```
import java.util.ArrayList;
import java.util.List;

// Observer interface
interface Party { void update(String reading); }

// Subject - provides the registration mechanism for all observers
class ClimateController {
  private List<Party> parties = new ArrayList<>();
  public void attach(Party p) { parties.add(p); }
  public void detach(Party p) { parties.remove(p); }
  protected void notifyParties(String reading) {
    for (Party p : parties) { p.update(reading); }  // pass message to all observers
  }
```

```
}

// Concrete subject - state change triggers notification
class GreenhouseSensorUnit extends ClimateController {
  private double temperature;
  public void setReading(double t) {
    temperature = t;
    if (temperature > 35.0) {
      notifyParties("ALERT: temperature " + temperature + " C exceeds threshold");
    }
  }
}

// Concrete observers
class DashboardDisplay implements Party {
  public void update(String reading) { System.out.println("[Dashboard] " + reading); }
}
class SmsAlertService implements Party {
  public void update(String reading) { System.out.println("[SMS] " + reading); } }
class EmailService implements Party {
  public void update(String reading) { System.out.println("[Email] " + reading); } }
class VentController implements Party {
  public void update(String reading) { System.out.println("[Vent] opening roof vents");
} }

public class Main {
  public static void main(String[] args) {
    GreenhouseSensorUnit sensor = new GreenhouseSensorUnit();
    sensor.attach(new DashboardDisplay());   // observers attached at run time
    sensor.attach(new SmsAlertService());
    sensor.attach(new EmailService());
    sensor.attach(new VentController());
    sensor.setReading(38.5);   // state change -> all parties updated automatically
  }
}
```

Marking: subject with observer collection + attach()/detach() [3]; notify loop calling update() on every registered observer [2]; observer interface with update() [1]; at least three concrete observers [2]; client attaching observers at run time and a state change triggering automatic notification [2]. Adapter alternative: interface ParcelHandler; class CustomsAdapter implements ParcelHandler holding a private CustomsClearanceLib reference set in the constructor, each method delegating (trackParcel() → lib.trackCustomsQR(), etc.); client uses ParcelHandler only — constructor injection [3], full delegation of all three methods [4], client demo [3].

TRAP TO WATCH: *The state-change trigger is the silent killer: if nothing CALLS notify() when the reading changes, the pattern is decorative and the demo marks vanish. In the adapter alternative, adding an 'extends CustomsClearanceLib' alongside the interface implementation contradicts the object-adapter you drew in (b).*

COVERAGE MAP — PAPER vs THE DOCTOR'S TIPS

Doctor's checklist item	Where tested	Source material
CDP: justify pattern choice vs alternatives	Q4(a) Builder-vs-AF decoys	Ch6 comparison tables (appear TWICE in new deck)
CDP: sketch class diagrams	Q4(b)	Ch6 GoF structures + FashionFactory/Order examples
CDP: translate diagram to Java code	Q4(d)	Ch6 taught idioms (factory delegation, ctor injection)
SDP & BDP: justify choice + category	Q5(a) + category trap	Ch7 GoF intents; only Adapter/Composite/Facade + Iterator/Observer taught
SDP & BDP: sketch class diagrams	Q5(b)	Ch7 attach/detach/notify/update naming; object adapter
SDP & BDP: Java code from diagram	Q5(d)	Ch7 notify-loop and delegation idioms
Management/Leadership/Ethics: Lab Exercise #4	Q1(b) EVM (Sara-style) + Q1(a) WBS/LRC	Ch9 Sara worked example; EVM appendix tables reproduced verbatim
Leadership & Ethics applied to real decisions	Q1(c) falsified-progress scenario	Ch9 SECE 8 principles; 4 leadership keys; Big Five (drill separately)
Construction: In-Class Exercise #1 (code from decision table)	Q2(c)+(d) EEDT + code excerpt	Ch8 phone-discount LEDT/EEDT worked examples
Construction: In-Class Exercise #2 (cyclomatic complexity)	Q3(b)+(c) two-approach cross-check	Ch8 three formulas: e−n+2p, π+1, regions+1; threshold 10
Construction: PDL	Q3(a)	Ch8 event-logger PDL example (indented natural language)
Construction: quality attributes	Q2(a)	Ch8: completeness, correctness, testability, maintainability

Not tested here but drill before the exam (recycled in past papers): Big Five dimensions applied to YOUR course project (Agreeableness, Surgency, Adjustment, Conscientiousness, Openness — prepare one concrete sentence each); WBS definition + TWO primary purposes as a theory question; state diagrams from a scenario (appeared in 2022/23); Composite and Facade as the 'other' structural options — know their GoF intents and the Adapter-vs-Facade discriminator (interface CONVERSION for one incompatible class vs a SIMPLIFIED UNIFIED interface over a whole subsystem, many-to-one).

TRAINING PROTOCOL (EXAM IS TOMORROW)

1. Sit this paper NOW under real conditions: 3 hours, closed book, diagrams by hand, EVM appendix only. Do Section B FIRST if patterns are your strength — 60 marks live there.

2. Self-mark with this scheme, entering part-scores per [bracket]. Anything below half marks on a part = re-do that part from scratch tonight, not re-read.

3. Retype Q4(d) and Q5(d) until they compile and run — pattern code must come out of your hand in under 12 minutes each.

4. Redraw the five taught pattern structures from memory (Factory Method, Abstract Factory, Builder, Adapter, Observer) with slide-exact participant names — 5 minutes each, check against the scheme.

5. Memorize cold: EVM equation table; π counting rules (compound = 2, switch = n−1, threshold 10); EEDT policy formula ∏Vi vs LEDT 2^n; the four construction quality attributes; the four leadership keys; five Big Five names; the four WBS/LRC codes and three WBS criteria.

6. Sleep. A 0.8 CPI brain at 9 a.m. loses more marks than any topic you could cram past midnight.